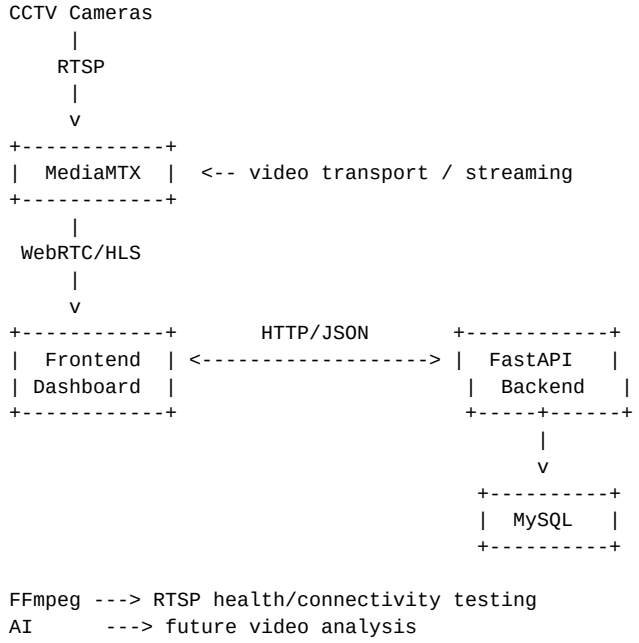
Gujarat Police Camera System

MediaMTX + FFmpeg + FastAPI + MySQL Setup Guide

A sequential setup and reference guide based on the camera-streaming system developed in our project.

1. What We Are Building

The system separates camera video transport from application management. RTSP cameras provide video to MediaMTX. FastAPI controls camera and stream operations and communicates with MySQL. FFmpeg is used mainly for RTSP connectivity/health testing. The browser receives a browser-friendly stream such as WebRTC from MediaMTX.



2. Roles of the Main Components

Component	Purpose	What it should NOT do
MediaMTX	Receive/pull RTSP and provide streaming protocols such as WebRTC/HLS.	Do not put application/database logic here.
FFmpeg	Test whether an RTSP URL can connect and produce frames; useful for health checks.	Do not use it as the main 80,000-camera video management layer.
FastAPI	API/control layer: cameras, vendors, start/stop, status, future events.	Do not carry every video frame through FastAPI.
MySQL	Store vendors, cameras, URLs, relationships, status and later events/logs.	Do not store the video stream itself.
Frontend	Police/operator dashboard showing cameras, status and vendor information.	Do not expose RTSP credentials in frontend code.

3. Recommended Project Structure

```
gujarat-police-camera-system/
|
+-- backend/
|   +-- app/
|       +-- main.py
|       +-- api/
|       |   +-- camera.py
|       |   +-- vendor.py
|       |   +-- stream.py
|       |   +-- health.py
|       |   +-- event.py
|       |   +-- auth.py
|       +-- services/
|       |   +-- camera_service.py
```

```
|       |   +-- vendor_service.py
|       |   +-- stream_service.py
|       |   +-- mediamtx_service.py
|       |   +-- health_service.py
|       |   +-- event_service.py
|       |   +-- auth_service.py
|       +-- database/
|       |   +-- connection.py
|       |   +-- camera_repository.py
|       |   +-- vendor_repository.py
|       |   +-- event_repository.py
|       |   +-- user_repository.py
|       +-- models/
|       +-- schemas/
|       +-- core/
|       +-- utils/
|
+-- frontend/
+-- database/
|   +-- schema.sql
|   +-- seed.sql
|   +-- migrations/
|
+-- streaming/
|   +-- mediamtx/
|   |   +-- mediamtx.yml
|   |   +-- README.md
|   +-- ffmpeg/
|       +-- README.md
|
+-- ai/
|   +-- detection/
|   +-- inference/
|   +-- models/
|   +-- tracking/
|
+-- scripts/
+-- docs/
+-- deployment/
+-- README.md
+-- .gitignore
```

4. Existing Windows Tools / Paths Used

During the prototype, MediaMTX and FFmpeg were installed outside the Git project folder. This is useful to remember because the executable location is different from the source-code repository.

```
FastAPI project:
D:\GIT_ml\multi_cam_setup

Main prototype file:
D:\GIT_ml\multi_cam_setup\api_for_cames.py

MediaMTX:
D:\mediamtx_v1.20.1_windows_amd64\
D:\mediamtx_v1.20.1_windows_amd64\mediamtx.exe
D:\mediamtx_v1.20.1_windows_amd64\mediamtx.yml

FFmpeg:
D:\ffmpeg-9.0.1-essentials_build\
D:\ffmpeg-9.0.1-essentials_build\ffmpeg-9.0.1-essentials_build\bin\
D:\ffmpeg-9.0.1-essentials_build\ffmpeg-9.0.1-essentials_build\bin\ffmpeg.exe
```

5. Setup Sequence

Step 1 — Create the project folder

```
mkdir gujarat-police-camera-system
cd gujarat-police-camera-system
```

Why: Why: This is the root of the Git/group project. Keep application code separate from installed streaming binaries.

Step 2 — Prepare Python environment

```
python -m venv .venv
.\.venv\Scripts\activate
```

Why: Why: A virtual environment prevents this project's Python packages from interfering with other projects.

Step 3 — Install backend packages

```
pip install fastapi uvicorn httpx mysql-connector-python
```

Why: Why: FastAPI provides the API server; Uvicorn runs it; HTTPX talks to MediaMTX's HTTP API; mysql-connector-python talks to MySQL.

Step 4 — Verify Python/FastAPI

```
python --version
pip --version
uvicorn --version
```

Why: Why: Confirm the environment is using the expected Python installation and that Uvicorn is available.

Step 5 — Verify FFmpeg

```
cd D:\ffmpeg-9.0.1-essentials_build\ffmpeg-9.0.1-essentials_build\bin
.\ffmpeg.exe -version
```

Why: Why: This confirms the correct modern FFmpeg executable is available.

Step 6 — Verify MediaMTX

```
cd D:\mediamtx_v1.20.1_windows_amd64
.\mediamtx.exe
```

Why: Why: Start MediaMTX and confirm that its configuration is valid. Run it from its own directory so relative paths/configuration behave predictably.

Step 7 — Enable MediaMTX Control API

```
api: true
apiAddress: :9997
apiEncryption: false
```

Why: Why: FastAPI needs a control interface to dynamically create/delete camera paths instead of manually writing every camera path into YAML.

Step 8 — Check MediaMTX Control API

```
Invoke-RestMethod "http://127.0.0.1:9997/v3/paths/list"
```

Why: Why: This checks whether FastAPI can reach the MediaMTX control API and shows the currently configured/active paths.

Step 9 — Test one RTSP source with FFmpeg

```
ffmpeg -rtsp_transport tcp -i "RTSP_URL" -t 5 -f null NUL
```

Why: Why: This tests the source itself. A successful connection is not enough; we want FFmpeg to actually receive/decode frames.

Step 10 — Test RTSP source through MediaMTX

```
POST /v3/config/paths/add/cam01
```

```
JSON:
{
  "source": "RTSP_URL",
  "rtspTransport": "tcp"
}
```

Why: Why: MediaMTX needs a path that maps a friendly stream name such as cam01 to the real RTSP source.

Step 11 — Test WebRTC

```
http://127.0.0.1:8889/cam01/
```

Why: Why: Browsers cannot normally consume the camera's RTSP URL directly. MediaMTX exposes a browser-friendly stream.

Step 12 — Start FastAPI

```
cd D:\GIT_ml\multi_cam_setup
uvicorn api_for_cames:app --reload
```

Why: Why: The backend must be running before the dashboard or API calls can control cameras.

Step 13 — Open FastAPI docs

```
http://127.0.0.1:8000/docs
```

Why: Why: FastAPI automatically provides interactive API documentation, useful for testing endpoints before connecting the frontend.

Step 14 — Dynamically start a camera

```
POST /camera/{camera_id}/start
```

Why: Why: FastAPI looks up the camera URL in MySQL and asks MediaMTX to create the stream path dynamically.

Step 15 — Dynamically stop a camera

```
POST /camera/{camera_id}/stop
```

Why: Why: This removes the MediaMTX path when the stream is no longer needed.

Step 16 — Check all camera health

```
POST /vendor/{vendorID}/check-cameras
```

Why: Why: FFmpeg tests each camera and the backend stores the resulting ONLINE/OFFLINE status.

Step 17 — Dashboard camera data

```
GET /cameras
```

Why: Why: The operator dashboard should retrieve all cameras, including VendorName and status, rather than forcing the operator to select a vendor.

6. MediaMTX YAML — Minimal Prototype Configuration

For the dynamic design, do NOT manually add cam01, cam02, ... cam80000 to the YAML. FastAPI creates/removes paths through the MediaMTX Control API. The YAML mainly enables the server and its interfaces.

```
# mediamtx.yml

api: true
apiAddress: :9997
apiEncryption: false

rtsp: true
rtspAddress: :8554

webrtc: true
webrtcAddress: :8889

hls: true
hlsAddress: :8888
```

Note: Exact option names can depend on the MediaMTX version. The above is the intended minimal concept for the v1.20.1 setup used in the prototype; keep the configuration shipped with your installed version as the authoritative baseline and change only what the project needs.

7. Dynamic MediaMTX Path

Instead of putting camera paths into YAML, FastAPI can call the MediaMTX Control API. For a camera whose database name is CAM01, the application can use the stream name cam01.

```
POST http://127.0.0.1:9997/v3/config/paths/add/cam01

JSON:
{
    "source": "RTSP_CAMERA_URL",
    "rtspTransport": "tcp"
}

Delete:
DELETE http://127.0.0.1:9997/v3/config/paths/delete/cam01

List:
GET http://127.0.0.1:9997/v3/paths/list
```

8. Important: RTSP Transport

The prototype used TCP explicitly for controlled testing. TCP is not automatically the fastest choice for every camera/network. UDP can have lower overhead, while TCP can be easier to operate through some networks because it avoids separate UDP transport issues. For a real deployment, measure the actual camera/network environment instead of assuming one transport is best for every vendor.

```
FFmpeg test:
ffmpeg -rtsp_transport tcp -i "RTSP_URL" -t 5 -f null NUL

MediaMTX dynamic path:
{
    "source": "RTSP_URL",
    "rtspTransport": "tcp"
}
```

9. FFmpeg Health Check

```
def check_camera(url):
    command = [
        "ffmpeg",
        "-rtsp_transport", "tcp",
        "-i", url,
        "-t", "5",
        "-f", "null",
        "-"
```

```
    ]

    try:
        result = subprocess.run(
            command,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            timeout=15
        )

        if result.returncode == 0:
            return "ONLINE"

        return "OFFLINE"

    except subprocess.TimeoutExpired:
        return "OFFLINE"
```

The 5-second test is the amount of stream time requested from FFmpeg. The 15-second subprocess timeout prevents a single unresponsive camera from blocking forever.

10. Concurrent Camera Health Checks

```
with ThreadPoolExecutor(max_workers=10) as executor:
    camera_results = executor.map(check_one_camera, cameras)
```

Why: If cameras are tested one-by-one, a slow camera can delay every camera after it. A worker pool lets several independent tests happen at the same time. The value 10 is a prototype setting; for a large deployment it must be load-tested and redesigned for scale.

11. Database Relationship

```
vendor
-----------------------------------------------
VendorID | VendorName | NUMBER_CAM | URL | enable
    |
    | 1-to-many relationship
    v
camera
-----------------------------------------------
CameraID | CameraName | VendorID | CameraURL | enable
```

VendorID connects a camera to its vendor. This lets the dashboard show all cameras together while displaying the vendor as metadata, such as a vendor-specific border/badge color.

12. Recommended All-Camera Dashboard API

```
GET /cameras

Example response:

[
  {
    "CameraID": 15,
    "CameraName": "CAM01",
    "VendorID": 1,
    "VendorName": "Vendor A",
    "status": "ONLINE"
  }
]
```

The operator should not need to select a vendor just to see cameras. FastAPI joins camera and vendor information, and the frontend uses VendorID/VendorName to visually identify the source vendor.

13. The Complete Runtime Flow

```
A. Operator opens dashboard
        |
        v
   GET /cameras
        |
        v
     FastAPI
        |
        v
      MySQL
        |
        v
 camera + vendor + status
        |
        v
    Dashboard
        |
        | operator starts CAM01
        v
 POST /camera/15/start
        |
        v
     FastAPI
        |
        | read CameraURL
        v
      MySQL
        |
        v
 MediaMTX Control API
        |
        v
   MediaMTX creates
     path "cam01"
```

```
        |
        v
 MediaMTX pulls RTSP
        |
        v
     Camera
        |
        v
 RTSP video -> MediaMTX
        |
        v
   WebRTC/HLS
        |
        v
     Browser


Health flow:

Camera URL
    |
    v
  FFmpeg
    |
  5 sec test
    |
    v
ONLINE / OFFLINE
    |
    v
  MySQL
    |
    v
Dashboard status
```

14. Troubleshooting Checklist

Problem	Check
ffmpeg not recognized	Run FFmpeg from its bin directory or add its bin directory to PATH.
MediaMTX API unavailable	Check that MediaMTX is running and api: true / apiAddress: :9997 are active.
Path already exists	Check GET /v3/paths/list. Stop/delete the existing path or make /start idempotent.
Browser says path undefined	Make sure the backend returns a real stream_name and the frontend uses it.
CORS error	Configure FastAPI CORS for the frontend origin during development.
RTSP connects but no frames	Test the exact RTSP URL directly with FFmpeg and VLC. A connected RTSP endpoint is not proof that usable vi
WebRTC connects but no video	Check whether MediaMTX has a video track and whether the source codec is browser-compatible.
FastAPI import error	Use the actual Python module name, e.g. uvicorn api_for_cames:app --reload for the prototype file.

15. Security Rules

• Do not commit real RTSP usernames/passwords to GitHub.

• Do not put RTSP credentials in frontend JavaScript or HTML.

• Use environment variables/secrets for database credentials and service credentials.

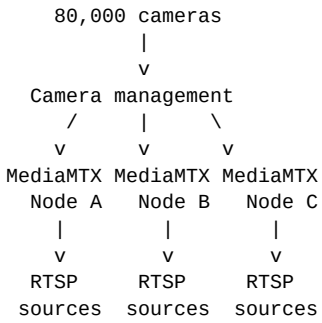
• The MediaMTX Control API should not be exposed publicly without appropriate protection.

• Rotate any real credential that was accidentally shared in public or team chats.

16. Scaling Note for 80,000 Cameras

The current prototype is intentionally simple. At 80,000 cameras, do not run all streams through one MediaMTX process or create a single giant YAML file. A production architecture would distribute streams across multiple

MediaMTX nodes/servers, use a service or scheduler to assign cameras to nodes, and monitor resource usage. The exact number of nodes depends on camera bitrate, codecs, connection behavior, hardware, network capacity and how many cameras are simultaneously viewed.




```
FastAPI + database remain the control/registry layer.
```

17. Recommended Build Order for the Group

1. Database schema and vendor/camera records.
2. FastAPI database connection.
3. GET /cameras returning camera + vendor information.
4. MediaMTX installation and minimal YAML.
5. MediaMTX Control API integration.
6. FastAPI start/stop camera endpoints.
7. FFmpeg health-check service.
8. Dashboard showing all cameras and vendor colors.
9. WebRTC/HLS integration for live viewing.
10. Logging, authentication, testing and deployment.
11. AI detection/tracking after the streaming foundation is stable.

18. Final Mental Model

```
Remember these five jobs:

MySQL    = remembers camera information
FastAPI  = controls the system
MediaMTX = handles video streaming
FFmpeg   = checks/tests camera streams
Frontend = shows everything to the operator

Do not make one component responsible for all five jobs.
```

This document is a project setup/reference guide, not a replacement for the configuration documentation shipped with the exact MediaMTX/FFmpeg versions being deployed.